

DEEP LEARNING

Assignment-2

Name: Subhapreet Patro

Roll No.: 2211CS010547

Section: I

Q1) Create a simple computational graph for the equation: $Y=(a+b)*(b-c)$ with explanation.

A)

To create a **computational graph** for the equation:

$$Y = (a + b) \times (b - c)$$

We break it down into smaller steps and represent each operation as a **node** in the graph. This is useful in deep learning and backpropagation because it lets us visualize and compute intermediate results and gradients efficiently.

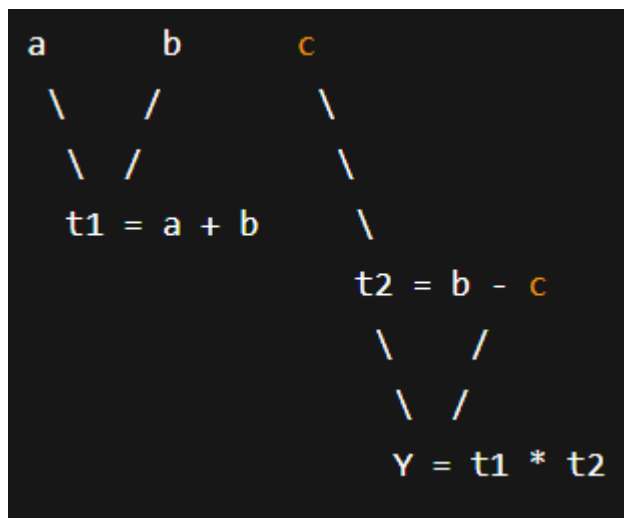
Let's define intermediate variables:

1. $t1 = a + b$
2. $t2 = b - c$
3. $Y = t1 \times t2$

Variables and Operations:

- Inputs: a, b, c
- Intermediate Nodes: $t1, t2$
- Output: Y

Computational Graph:



Explanation of Each Node:

Node	Operation	Description
t1	$a + b$	Adds inputs a and b
t2	$b - c$	Subtracts c from b
Y	$t1 \times t2$	Multiplies the results of t1 and t2

Example with Values:

Let's take:

- $a = 2, b = 3, c = 1$

Compute step-by-step:

1. $t1 = a + b = 2 + 3 = 5$
2. $t2 = b - c = 3 - 1 = 2$
3. $Y = t1 \times t2 = 5 \times 2 = 10$

Final Output:

$Y = (a + b) \times (b - c)$ is represented as a graph with:

- **Input nodes:** a, b, c
- **Operation nodes:** addition, subtraction, multiplication
- **Output node:** Y

This structure is useful for:

- Forward computation (to calculate the value of Y)
 - Backward computation (to calculate gradients with respect to a, b, c in machine learning)
-

Q2) Explain Perceptron Activation Functions: Sigmoid

A)

What is a Perceptron?

A **perceptron** is the basic unit of a neural network. It takes multiple inputs, applies weights to them, sums them up, and then passes the result through an **activation function** to produce an output.

Activation Functions

Activation functions introduce **non-linearity** into a neural network. Without them, the entire network would behave like a linear model, no matter how many layers it has.

The **Sigmoid** function is one of the most commonly used activation functions—especially in early neural networks and binary classification problems.

Sigmoid Activation Function

► Mathematical Formula:

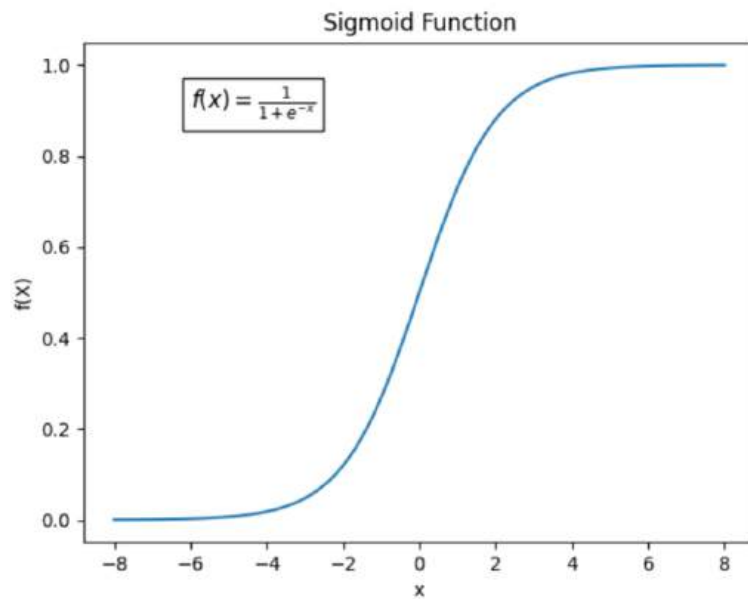
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Where:

- x is the input to the activation function (the weighted sum of inputs),
- $\sigma(x)$ is the output, always between 0 and 1.

Sigmoid Graph

- The output of sigmoid smoothly ranges from **0 to 1**.
- It's **S-shaped** (also called a **logistic curve**).



x (input)	sigmoid(x)
$-\infty$	~ 0
0	0.5
$+\infty$	~ 1

Sigmoid in a Perceptron

1. Inputs: $x_1, x_2, \dots, x_n$
2. Weights: $w_1, w_2, \dots, w_n$
3. Weighted sum:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

4. Activation (Sigmoid):

$$y = \frac{1}{1 + e^{-z}}$$

Properties of Sigmoid

Property	Description
Range	(0, 1)

Non-linear	Helps neural networks learn complex patterns
Differentiable	Smooth and has a well-defined derivative
Derivative	

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Used in backpropagation |

| **Centered** | **Not zero-centered**, which may slow convergence |

Advantages

- Useful for **binary classification** (acts like probability).
- Smooth gradient — helps in **gradient-based optimization**.
- Output is always bounded between 0 and 1.

Disadvantages

Issue	Explanation
Vanishing gradient	For very high or low input values, gradients become very small → slow learning
Not zero-centered	Can lead to zigzagging gradients during optimization
Computationally expensive	Exponential function $e^{-x}e^{\{x\}}$ is more expensive than ReLU

Use Cases

- **Output layer** in **binary classification models**
- Logistic regression
- Earlier neural network models

Q3) Build a Simple model for predicting output of the XOR logic gate using TensorFlow.

A)

What is XOR?

The **XOR (Exclusive OR)** logic gate has the following behavior:

Input A	Input B	Output ($A \oplus B$)
0	0	0

0	1	1
1	0	1
1	1	0

The XOR function is **not linearly separable**, so we need a neural network with **at least one hidden layer** to solve it.

Steps to Build the XOR Model Using TensorFlow

We'll use TensorFlow and Keras to build a small neural network.

1. Install Required Libraries

If you haven't already:

```
pip install tensorflow
```

2. Import TensorFlow

```
import tensorflow as tf
```

```
import numpy as np
```

3. Prepare the Data

```
# Input features
```

```
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=np.float32)
```

```
# XOR outputs
```

```
y = np.array([
    [0],
    [1],
    [1],
    [0]
])
```

```
[1],  
[0]  
, dtype=np.float32)
```

4. Build the Model

```
model = tf.keras.Sequential([  
    tf.keras.layers.Dense(4, input_shape=(2,), activation='sigmoid'),  
    tf.keras.layers.Dense(1, activation='sigmoid')  
)
```

Explanation:

- **Input layer:** Takes 2 inputs (A and B)
- **Hidden layer:** 4 neurons with **sigmoid** activation
- **Output layer:** 1 neuron with **sigmoid** activation (for binary classification)

5. Compile the Model

```
model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

6. Train the Model

```
history = model.fit(X, y, epochs=1000, verbose=0)
```

7. Evaluate the Model

```
loss, accuracy = model.evaluate(X, y)  
print(f"Loss: {loss:.4f}, Accuracy: {accuracy:.4f}")
```

8. Make Predictions

```
predictions = model.predict(X)
predicted_labels = (predictions > 0.5).astype(int)

print("Predictions:")
for i in range(len(X)):
    print(f"Input: {X[i]} → Predicted: {predicted_labels[i][0]}, Actual: {int(y[i][0])}")
```

Expected Output:

```
Input: [0. 0.] → Predicted: 0, Actual: 0
Input: [0. 1.] → Predicted: 1, Actual: 1
Input: [1. 0.] → Predicted: 1, Actual: 1
Input: [1. 1.] → Predicted: 0, Actual: 0
```

Q4) What is a Perceptron & explain its components with a diagram?

A)

What is a Perceptron?

A **Perceptron** is the **simplest type of artificial neural network**, introduced by **Frank Rosenblatt** in 1958. It models how a single neuron in the brain might process inputs and produce an output. It's primarily used for **binary classification** (classifying input into one of two categories).

Definition

A **perceptron** takes multiple inputs, applies corresponding weights, adds a bias, and passes the result through an **activation function** to produce a binary output (usually 0 or 1).

Mathematical Representation

For inputs: $x_1, x_2, \dots, x_n$

Weights: $w_1, w_2, \dots, w_n$

Bias: b

Weighted Sum:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Output:

$$y = \text{activation}(z)$$

If using a **step function**, the output is:

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Components of a Perceptron

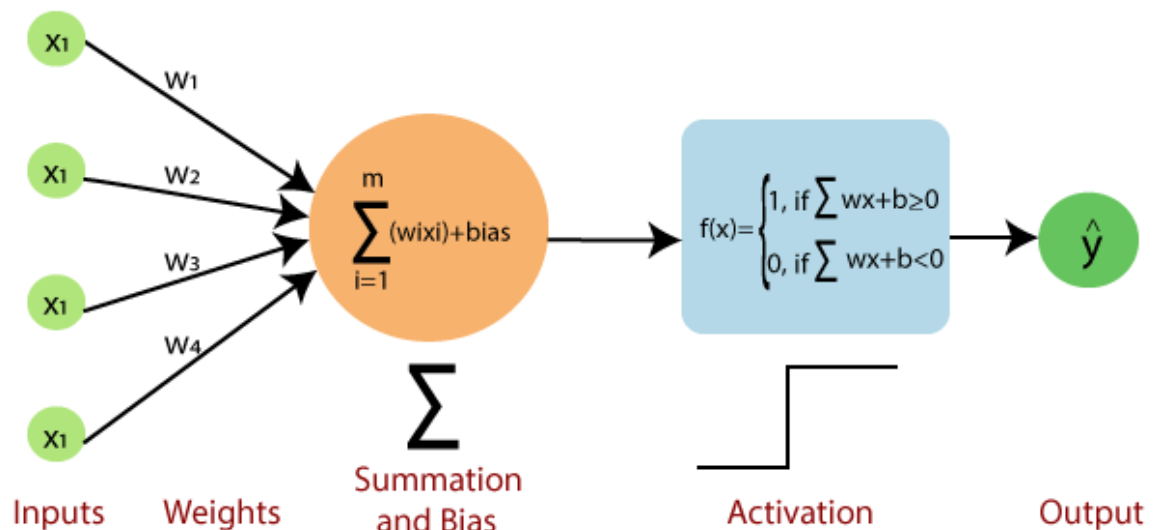
Component	Description
Inputs ($x_1, x_2, \dots, x_n$)	Numerical features fed into the model
Weights ($w_1, w_2, \dots, w_n$)	Coefficients that determine the importance of each input
Bias (b)	A constant added to the weighted sum for shifting the activation
Summation Function	Adds all weighted inputs and the bias
Activation Function	Converts the sum into a binary output (like step, sigmoid, ReLU, etc.)
Output (y)	Final prediction (0 or 1 for binary classification)

Working of a Perceptron – Step by Step

1. Receive inputs $x_1, x_2, \dots, x_n$
2. Multiply each input by its weight $w_1, w_2, \dots, w_n$

3. Add the results + bias → total sum zz
4. Pass zz through an **activation function**
5. Output the final result yy

Perceptron Diagram



Example

Let's say:

- $x_1=2, x_2=3$
- $w_1=0.4, w_2=0.7$
- $b=-1$

Then,

$$z = (2 \times 0.4) + (3 \times 0.7) + (-1) = 0.8 + 2.1 - 1 = 1.9$$

If using a step function:

$$y = 1 \text{ (since } 1.9 > 0 \text{)}$$

Learning in a Perceptron

During training:

- The perceptron updates its **weights** and **bias** using **gradient descent** and an error term to minimize incorrect predictions.

Limitations of Single-Layer Perceptron

Limitation	Explanation
Only linearly separable data	Cannot solve problems like XOR
No hidden layers	Cannot learn complex patterns

To overcome this, we use **multi-layer perceptrons (MLPs)**, which are the building blocks of **deep neural networks**.